Week 02 Report: Capacity-Bound, Back-Invalidation and Write-Intensive L2 Workloads

This week the three assigned L2 workloads (capacity-bound, back-invalidation thrashing and write-intensive) were implemented as modes of one C++ generator, each with a prediction of its L2 behaviour, and run on a revised gem5 model of a 4-core CVA6-like cluster. The first capacity-bound point matched its prediction exactly: with 8 of 16 L2 ways active and a 768 KiB footprint, the L2 demand miss rate was 1.000, and all 427,359 L2 demand accesses were accounted for by L1 data-cache misses. Back-invalidation and write-intensive runs are prepared, but each depends on one specification decision.

1. Objectives

The week had four objectives, all serving the project's first sweep priority, the number of active L2 ways for the way-shutdown proposal (P1).

1. Bring the gem5 cluster script, `cva6_cluster_baseline.py`, into agreement with the CVA6 RTL and with the team's frozen base model, and record the source of every parameter.
2. Make the L2 geometry express way shutdown correctly: fewer active ways at a constant set count, rather than a smaller cache with a different set count.
3. Replace the placeholder DSP program with a workload generator, `l2_workloads.cpp`, whose access patterns produce predictable L2 behaviour for the three workloads assigned this week: capacity-bound, back-invalidation thrashing and write-intensive.
4. Run one configuration end to end and check the result against a prediction made before the run.

2. System model

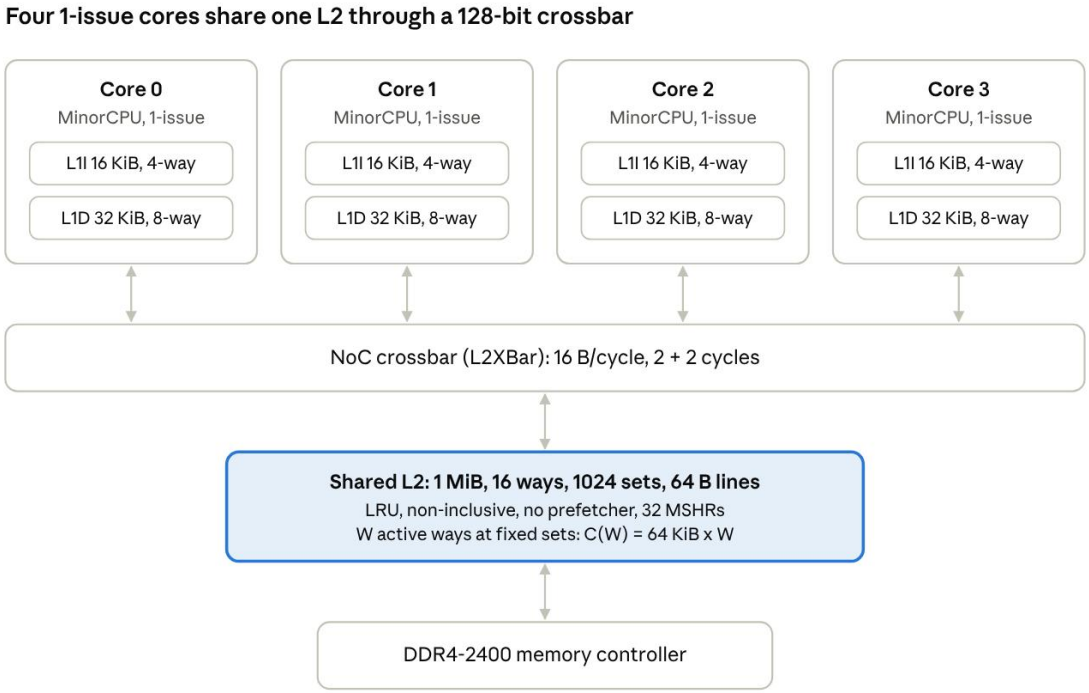
The model is a 4-core cluster simulated in gem5 25.1.0.1 (RISC-V build, classic memory system, syscall-emulation mode) at 1 GHz. Each core is a `RiscvMinorCPU` tuned towards CVA6, with private L1 instruction and data caches. The L1 caches connect through a crossbar (`L2XBar`) standing in for the NoC to a shared L2, which connects through a system crossbar to a single DDR4 memory controller with a 512 MiB address range.

Component	Parameter	Value	Label	Initial draft
Core	Issue width (`decodeInputWidth`, `executeInputWidth`, `executeIssueLimit`)	1	RTL (`SuperscalarEn = 0`)	2 (MinorCPU default)
Core	Commit limit (`executeCommitLimit`)	2	RTL (`NrCommitPorts = 2`)	2 (default)
Core	`decodeToExecuteForwardDelay`	2 cycles	APPROX (stands in for the CVA6 issue stage)	2

Component	Parameter	Value	Label	Initial draft
Core	Branch predictor	gem5 default	Open (CVA6: BHT 128, BTB 32, RAS 2)	default
L1I	Size, associativity	16 KiB, 4-way	RTL	32 KiB, 4-way
L1D	Size, associativity	32 KiB, 8-way	RTL	32 KiB, 4-way
L1I, L1D	Tag / data / response latency	1 / 1 / 1 cycles	BASE	1 / 1 / 1
L1I, L1D	MSHRs, targets per MSHR	1, 8	BASE (CVA6 has one read MSHR per L1)	4, 8
L1I, L1D	Write buffers	8	RTL (`WtDcacheWbufDepth = 8`)	default
L1D	Write policy	Write-back	APPROX (RTL default is write-through)	write-back
All caches	Line size	64 B	SPEC (RTL L1 line is 16 B)	64 B
NoC	Width	16 B/cycle (128-bit)	BASE	32 B/cycle (256-bit)
NoC	Frontend / forward / response latency	2 / 2 / 2 cycles	BASE	2 / 2 / 2
L2	Production capacity, ways, sets	1 MiB, 16-way, 1024 sets	SPEC	1 MiB, 16-way
L2	Replacement	LRU (`LRURP`)	SPEC	not set
L2	Clusivity, `writeback_clean`	`mostly_incl` (non-inclusive), false	SPEC	default
L2	Prefetcher	None	SPEC	none
L2	Tag / data / response latency	14 / 14 / 14 cycles	To be calibrated	14 / 14 / 14
L2	MSHRs, targets, write buffers	32, 12, 16	BASE (fixed; MSHR count is out of scope)	32, 12, 16

Component	Parameter	Value	Label	Initial draft
Memory	DRAM model	DDR4_2400_8x8	BASE	DDR5_8400_4x8

The last column records the initial draft of the script. The issue width, the L1 sizes and associativities, the L1 MSHR count, the NoC width and the DRAM model were all changed this week to match the RTL or the base model.



Simulated cluster · 4 cores, private L1s, crossbar, shared L2, DDR4

Every L1 miss crosses the same crossbar into the one shared L2, so the L2 sees the interleaved miss streams of all four cores; way shutdown changes only the number of active ways inside the highlighted L2.

3. L2 geometry and sweep modes

Way shutdown removes ways but keeps the set index, so a cache with W active ways is modelled as a W-way cache with the production set count. The initial draft exposed only `--l2_size` and `--l2_assoc`, which change the set count whenever capacity changes and therefore cannot represent way shutdown. The revised script derives the geometry from the production capacity:

$$S = \frac{C_{prod}}{64 \cdot 16} = 1024, \qquad C(W) = S \cdot 64 \cdot W = 64\,\mathrm{KiB} \cdot W$$

The script supports the project's three sweeps through separate options.

Sweep	Options	Held constant	Varied	Set count
(1) Active ways	`--l2-active-ways W` (default 16), with `--l2-prod-size`	Sets, line size	W from 1 to 16	1024

Sweep	Options	Held constant	Varied	Set count
	(default 1 MiB)			
(2) Capacity	`--l2-size C`	16 ways, line size	C	C / 1 KiB
(3) Associativity	`--l2-size C --l2-assoc A`	Capacity, line size	A	C / (64 · A)

For sweep (1), the effective capacity of the active-way points is shown below. The total capacity of the eight L1 caches is 4 × (16 + 32) KiB = 192 KiB, which matters for the back-invalidation workload in Section 5.2.

Active ways W	Effective L2 capacity
16	1024 KiB
13	832 KiB
12	768 KiB
10	640 KiB
8	512 KiB
4	256 KiB
2	128 KiB

The script rejects a W outside 1 to 16 and any geometry whose set count is not a power of two, because gem5's set-associative indexing assumes one. With LRU replacement, a W-way cache at a fixed set count behaves exactly like the production cache with 16 − W ways disabled; for tree-PLRU the equivalence would only be approximate.

4. Simulation methodology

Processes. The simulation runs four independent copies of the workload program, one per core, as separate SE processes (PIDs 100 to 103). The script passes each copy its mode, its share of the footprint, the stride, its core index, a seed (`--seed` + core index) and an iteration count of 0, which makes the program loop indefinitely. The `--footprint` option is the aggregate across all four cores; each core receives one quarter. In the initial draft every core received the full footprint, so the aggregate was four times the stated value.

Measurement window. In the initial draft, statistics covered the whole run, including program initialisation and the tail in which some cores had finished and the L2 was no longer shared by four cores. The base-model study had already shown that such timing effects can move the miss rate near the L2 capacity from 0.07 to 0.46. The revised script therefore simulates in two phases:

1. A warm-up of `--warmup-cycles` cycles (default 2,000,000), after which all statistics are reset with `m5.stats.reset()`.
2. A measurement window of `--measure-cycles` cycles (default 10,000,000, i.e. 10 ms of simulated time at 1 GHz), after which statistics are dumped.

Because the programs never terminate, each phase must end with the cause `simulate() limit reached`. Any other cause, such as a process exiting, aborts the run with an error, so a run whose window is contaminated by a finished core cannot produce a result silently.

Statistics extraction. After the dump, the script writes two files to the output directory. `run_params.json` stores all command-line options together with the derived L2 size, associativity, set count, sweep type and aggregate footprint. `summary.json` stores the parameters and the selected statistics. Statistics are matched with regular expressions instead of fixed names, because names differ between gem5 releases:

- L2 demand accesses, hits, misses and miss rate (total and per requestor), and L2 writebacks
- L1D demand misses, accesses and writebacks, and L1I demand misses, per core
- per-core IPC and committed instructions
- DRAM average access latency, bus utilisation, read and write requests

A pattern that matches nothing is reported on the console. The initial draft wrote a CSV into each run's own output directory, which could not be merged across runs; collation is now left to the team's sweep script.

Sanity check. The script compares L2 demand accesses with the sum of L1I and L1D demand misses over the four cores. Their ratio should be close to 1, because every L2 demand access originates from an L1 miss and the L2 has no other requestor.

5. The three assigned workloads

Three workloads were assigned this week: capacity-bound, back-invalidation thrashing and write-intensive. Each is implemented as a mode of one generator, `l2_workloads.cpp`, and each is designed so that its L2 behaviour can be predicted before simulation, which turns every run into a test that can fail.

Workload	Modes	What it isolates	Status this week
Capacity-bound	`cap`, `cap_rand`	Miss rate as a function of footprint against active L2 capacity C(W)	Implemented, host-tested, first gem5 point validated
Back-invalidation thrashing	`binv`	L1 misses caused by L2 evictions of L1-resident lines	Implemented and host-tested; not yet simulated
Write-intensive	`write`, `rmw`	Dirty-line writeback traffic L1 → L2 → DRAM	Implemented and host-tested; not yet simulated

Common implementation. The generator is a single C++17 program, built with `riscv64-linux-gnu-g++ -std=c++17 -O2 -static` and invoked as `l2_workloads <mode> <bytes_per_core> <stride> <core_id> <seed> <iters>`.

- The buffer is page-aligned (`aligned_alloc(4096, bytes)`) and owned by a `unique_ptr` with a `free` deleter.
- A `memset` touches every byte before the selected mode starts. In SE mode gem5 assigns a physical page on first touch, so all page allocation happens before the measured phase.
- Results accumulate into a `volatile` global, so the compiler cannot remove loops whose results are otherwise unused.

- All modes share one loop template, `repeat(iters, body)`; `iters = 0` loops forever, and the gem5 script ends the run with its measurement window.
- Invalid input (a stride that is not a multiple of 8 B, a footprint below 128 B, a `binv` footprint not larger than its hot region) exits with status 2, so a misconfigured process fails visibly.

On x86-64, the C and C++ versions produced identical checksums for `cap`, `binv`, `write` and `rmw` with a 1 MiB buffer, a 64 B stride and three passes (49,152; 438,912; 2; 7). Each value equals the one computed by hand from the loop structure.

5.1 Capacity-bound workload

Purpose. This workload answers the central question of way shutdown: how far the active capacity C(W) can be reduced before the working set no longer fits in the L2.

Access pattern. `cap` reads the buffer sequentially at the given stride and repeats the sweep indefinitely. `cap_rand` stores in each line a pointer to the next line of a random cyclic permutation (Fisher–Yates shuffle, `std::mt19937_64`, seeded from the command line) and then chases the pointers. Every load in `cap_rand` depends on the previous one, so each miss exposes its full latency and no stride regularity is visible to the memory system.

Prediction. Both modes reuse every line once per pass, in the same order each pass, so the reuse distance of every line equals the whole footprint F. Under LRU this gives a cliff:

- $F \leq C(W)$: after warm-up every access hits in the L2, so the L2 miss rate is near 0.
- $F > C(W)$: every line is evicted shortly before its reuse, so the L2 miss rate is near 1.

The cliff applies to `cap_rand` as well, because its random order is repeated identically every pass. A smooth miss-rate curve would require independent random accesses, for which the expected miss rate is about $1 - C(W)/F$ when $F > C(W)$. That mode is not yet implemented. The code comment in the current version, which says `cap_rand` has no LRU cliff, is incorrect and will be fixed.

Metrics. L2 demand miss rate, per-core MPKI, and L1D demand misses for the accounting check.

5.2 Back-invalidation thrashing

Purpose. In an inclusive L2, evicting a line also invalidates its copies in the L1 caches. Lines that are hot in an L1 hit there and never reach the L2, so the L2's LRU state is never refreshed for them. The L2 therefore tends to evict exactly the lines the cores use most, and each such eviction becomes an L1 miss. Way shutdown makes this worse, because fewer ways mean more evictions. An inclusive L2 also sets a hard floor on W: C(W) must at least hold the 192 KiB of L1 contents, so $W \geq 3$, and more in practice.

Access pattern. `binv` reserves an 8 KiB hot region, which fits easily in the 32 KiB L1D, and streams through the rest of the buffer as cold data. For every cold line read, it reads 8 hot lines in round-robin order.

Prediction. With a non-inclusive L2, the hot region stays in the L1D, so L1D misses should equal the number of cold lines accessed, (bytes − 8 KiB) / stride per pass and core. With an inclusive L2, every L1D miss above that count is a back-invalidation.

Metric. L1D demand misses compared with the cold-line count; the excess measures back-invalidation.

Constraint. gem5's classic caches offer only `mostly_incl` and `mostly_excl`, neither of which enforces inclusion, and the proposed specification is non-inclusive. In the current model this workload can

therefore only serve as a control, expected to show zero excess. Measuring the effect requires either confirmation that the production L2 is inclusive, followed by a Ruby model with an inclusive L2 (for example `MESI_Two_Level`), or a decision that back-invalidation is out of scope.

5.3 Write-intensive workload

Purpose. This workload measures dirty-data traffic: L1D writebacks into the L2, L2 writebacks into DRAM, and the number of dirty lines resident in the L2. The last quantity sets the transition cost of way shutdown, because dirty lines in the ways being disabled must be written back first.

Access pattern. `write` performs one 64-bit store per stride position with no loads, like `memset`. `rmw` loads, increments and stores each word, like an in-place update.

Prediction (64 B stride, per-core footprint larger than the 32 KiB L1D):

- L1D writebacks are about one per line per pass, because every line is dirtied and later evicted.
- F > C(W): every dirty line is also evicted from the L2 each pass, so DRAM write requests are about one per line per pass. Because gem5 caches write-allocate, each store miss also fetches the line, so DRAM reads are about one per line per pass too.
- F ≤ C(W): dirty lines stay in the L2, so DRAM traffic falls to near zero in steady state, while the L2 holds up to F / 64 dirty lines.

Metrics. L1D writebacks, L2 writebacks, DRAM read and write requests.

Constraint. The default CVA6 configuration has a write-through L1D, whereas gem5 classic models write-back only and the base model uses the `_wb` variant. Under write-through every store reaches the L2, so at an 8 B stride the number of L2 write transactions would be up to eight times higher than in this model. This choice must be fixed, or stated as an approximation, before the results are reported.

6. Capacity-bound workload: first result in gem5

First gem5 run. The first configuration was capacity-bound mode `cap` with a 768 KiB aggregate footprint (192 KiB, or 3,072 lines, per core), a 64 B stride and 8 active ways, so C(8) = 512 KiB. Because the footprint exceeds the active capacity, a cyclic sweep under LRU evicts every line before its reuse, and the prediction made before the run was an L2 demand miss rate close to 1.

```
build/RISCV/gem5.opt -d m5out/cap_W8_F768K my_project/baseline/cva6_cluster_baseline.py \
  --binary my_project/baseline/l2_workloads --workload cap --footprint 768KiB --l2-active-ways 8
```

Statistic (measurement window, 10 M cycles)	Value	Label
L1D demand misses, core 0	106,974	Measured
L1D demand misses, core 1	106,687	Measured
L1D demand misses, core 2	106,946	Measured
L1D demand misses, core 3	106,752	Measured
Sum of L1D demand misses	427,359	Computed
L2 demand accesses	427,359	Measured

Statistic (measurement window, 10 M cycles)	Value	Label
L2 demand misses	427,359	Measured
L2 demand miss rate	1.000	Measured
L1I demand misses, L2 writebacks	0 (absent from `stats.txt`)	Measured
Mean cycles between L1D misses per core	93.6	Computed

The result supports three conclusions.

1. **The prediction held.** The L2 demand miss rate was exactly 1.000, as expected for a cyclic sweep larger than the LRU capacity.
2. **The accounting is exact.** L2 demand accesses equal the sum of L1D demand misses to the last access (ratio 1.000). Instruction misses and writebacks were zero, as expected for a read-only loop whose code stays resident in the L1I after warm-up. No L2 traffic is unaccounted for.
3. **All four cores ran throughout the window.** The per-core miss counts differ by at most 287, or 0.27 % of their mean. A core that had stalled or exited would show a markedly lower count. The mean interval of 93.6 cycles per miss is consistent with the DRAM pointer-chase latency of 85.6 to 87.3 cycles measured for the base model, plus loop overhead.

This run confirms only one side of the capacity cliff, the one where the cache is too small. The side where the footprint fits, and the boundary between them, remain to be tested.